UNIVERSIDAD
Popular del cesar

Ingeniería de Sistemas



IMPLEMENTACION DE PATRON FACADE EN RUMBANG

HECTOR ARTURO GUERRA PEREZ
MAIKEL ALEXIS PAYARES DOMINGUEZ

Facultad de Ingeniería y Tecnología

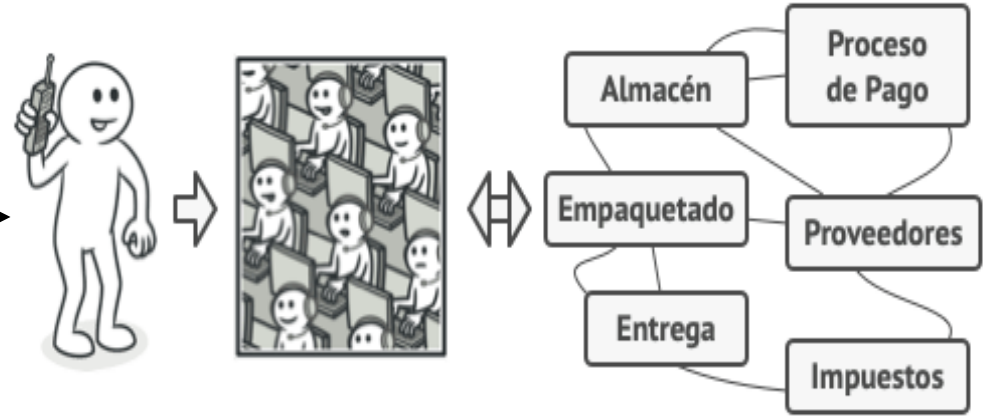
Valledupar 2026-2

FACADE - FACHADA

Una fachada es una clase que proporciona una interfaz simple a un subsistema complejo que contiene muchas partes móviles. Una fachada puede proporcionar una funcionalidad limitada en comparación con trabajar directamente con el subsistema. Sin embargo, tan solo incluye las funciones realmente importantes para los clientes

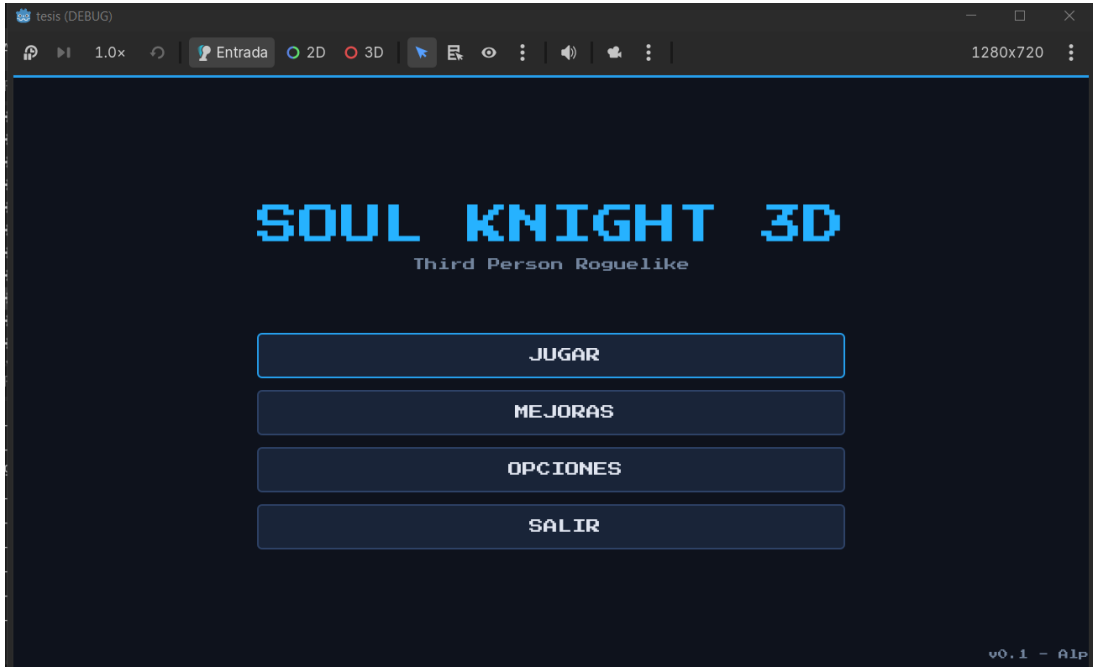
cuando llamas a una tienda para hacer un pedido por teléfono, un operador es tu fachada a todos los servicios y departamentos de la tienda. El operador te proporciona una sencilla interfaz de voz al sistema de pedidos, pasarelas de pago y varios servicios de entrega.

Analogía en el mundo real



Haciendo pedidos por teléfono.

APLICABILIDAD



- - Cuando se requiere una interfaz simple y de alto nivel para un subsistema complejo.
- - Cuando existen múltiples dependencias cruzadas entre el cliente y varias clases.
- - Cuando se desea estratificar o encapsular un subsistema para facilitar su
- mantenimiento o migración sin romper el código cliente.

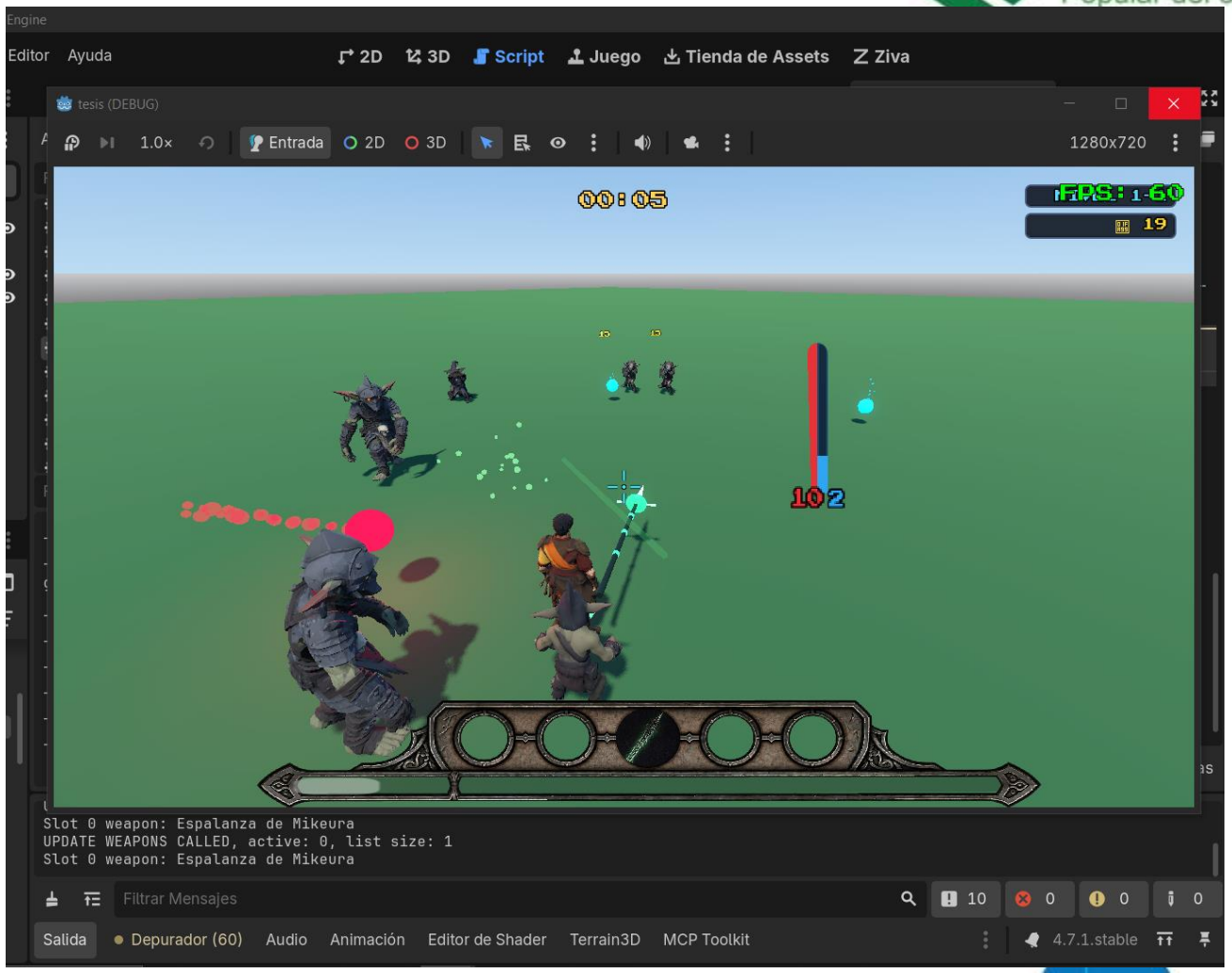
CONTEXTO DEL PROYECTO

- Juego de acción/supervivencia 3D en Godot desarrollado con múltiples subsistemas:

UI/HUD, Entorno/Terreno procedural, Oleadas de enemigos, Sistema de Botín (Loot)

y Adaptación por plataforma (Móvil vs Escritorio).

- La escena principal (Main) funciona como punto de entrada y ciclo de vida de la partida.

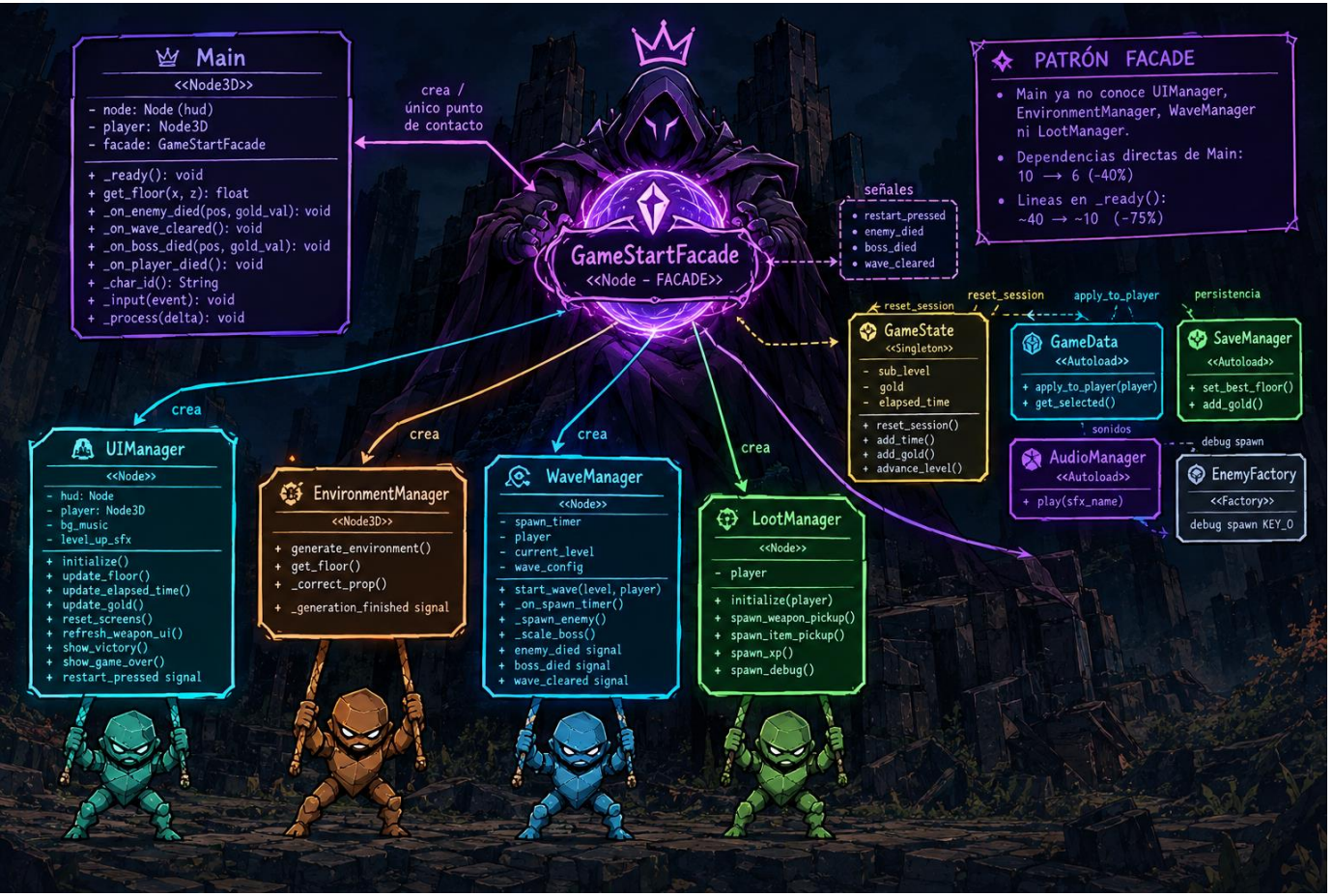


MOTIVACION/PROBLEMA

- "God Class" y "God Method": La clase Main y su método _ready() concentraban
demasiadas responsabilidades (violación del principio SRP).
- Alto Acoplamiento: Main debía conocer, instanciar, ordenar y conectar señales
manualmente de 5 gestores distintos (UIManager, WaveManager, LootManager,
EnvironmentManager y PlatformFactory).
- Fuga de Abstracción y Fragilidad: Main manipulaba directamente rutas duras ($Nodo)
y variables internas del Player (calcular altura Y, resetear vida/escudo/energía
y disparar señales internas a mano).



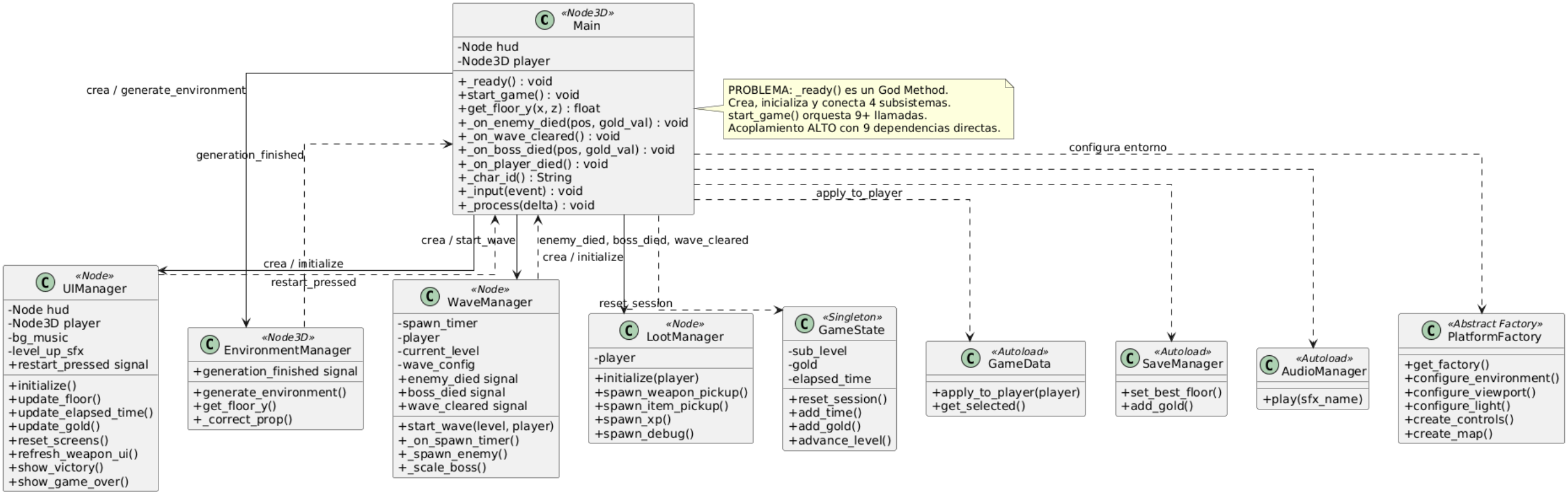
PROPOSITO



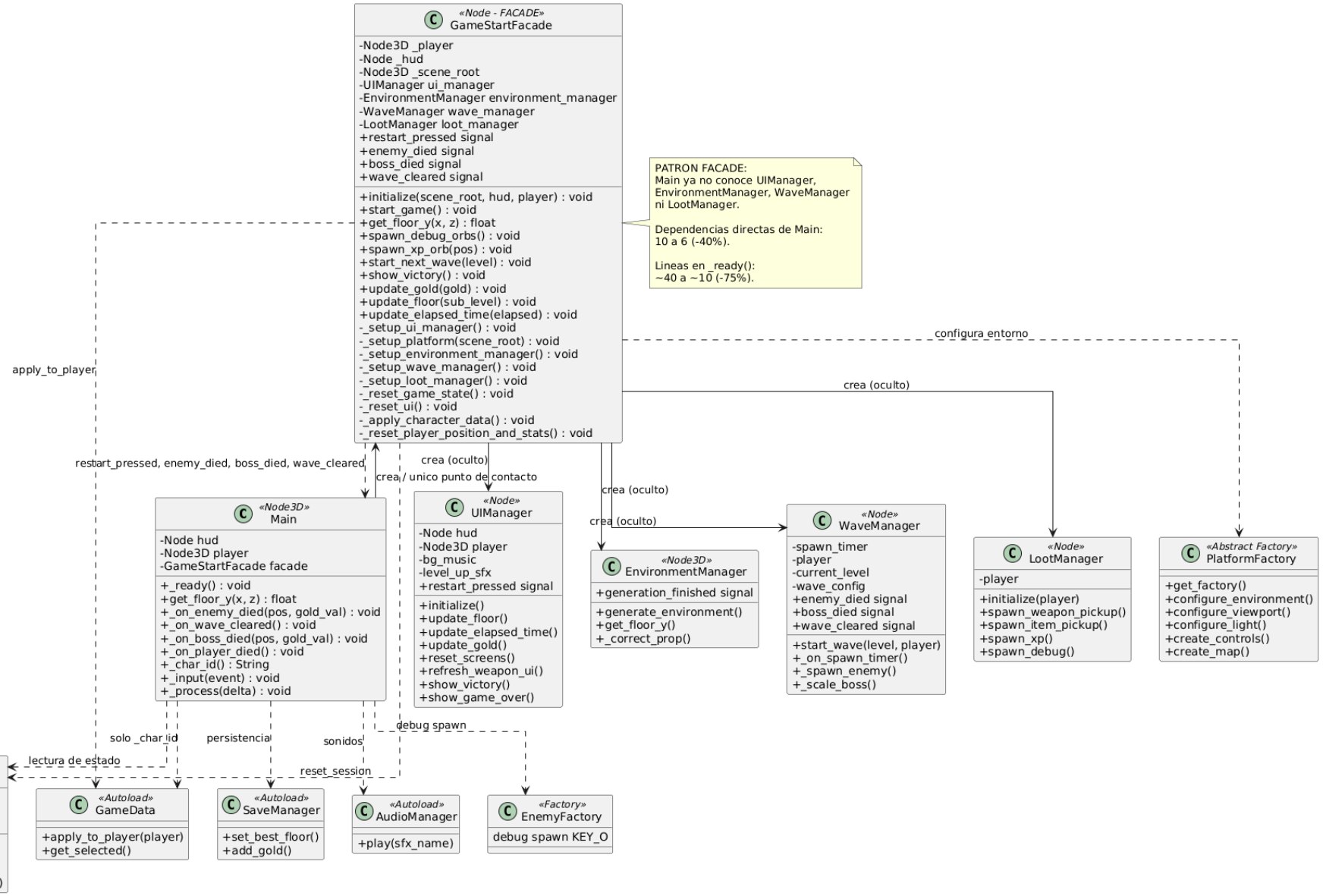
- Punto de Acceso Unificado: Introduce la clase 'GameStartFacade' como ventanilla
única entre Main y todos los subsistemas.
- Desacoplamiento Total: Main ya no conoce los detalles internos de los 5 managers;
solo le pide a la fachada acciones de alto nivel (initialize, start_game, start_next_wave).
- Secuencia de Inicio Consistente: Centraliza y garantiza el orden estricto de arranque
y reinicio de la partida (reset de estado -> UI -> spawn de jugador -> oleada -> loot).
- Código Limpio y Mantenible: Main pasa de un monolito acoplado a un controlador
conciso de eventos (~80 líneas) enfocado exclusivamente en el flujo del juego.

Modelos del sistema - ANT

UNIVERSIDAD Popular del cesar



Visualización - ACT



CODIGO

UNIVERSIDAD Popular del cesar

sin cambios

```
# [MAL APORTADO / ALTO ACOPLAMIENTO 3]: Generación de entorno.
# Main gestiona directamente el ciclo de vida de EnvironmentManager.
var env_manager = EnvironmentManager.new()
env_manager.name = "EnvironmentManager"
add_child(env_manager)
env_manager.generation_finished.connect(start_game)
env_manager.generate_environment()

# [MAL APORTADO / ALTO ACOPLAMIENTO 4]: Gestión de oleadas y eventos de comba
# Main tiene que conectar y conocer individualmente cada señal de WaveManager
var wave_manager = WaveManager.new()
wave_manager.name = "WaveManager"
add_child(wave_manager)
wave_manager.enemy_died.connect(_on_enemy_died)
wave_manager.boss_died.connect(_on_boss_died)
wave_manager.wave_cleared.connect(_on_wave_cleared)

# [MAL APORTADO / ALTO ACOPLAMIENTO 5]: Sistema de economía y pickups (LootMa
# Otro manager más que Main debe conocer e inicializar explícitamente.
var loot_manager = LootManager.new()
loot_manager.name = "LootManager"
add_child(loot_manager)
loot_manager.initialize(player)
```

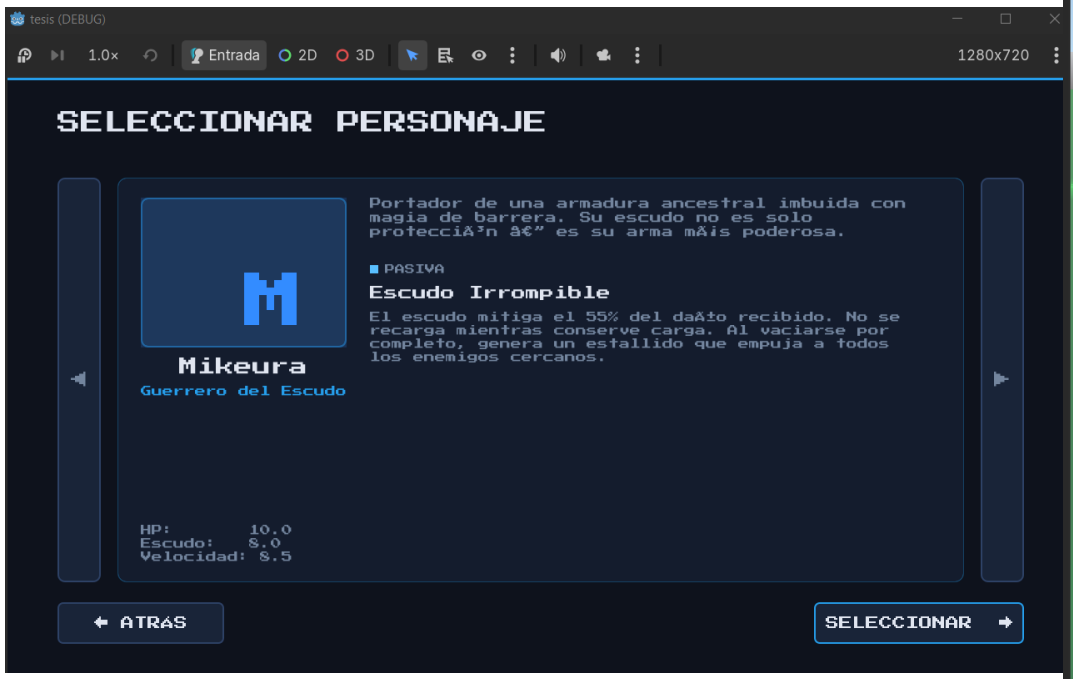
con cambios

```
func _ready() -> void:
    # --- APLICACIÓN DEL PATRÓN FACADE ---
    # En lugar de crear y conectar 4+ managers manualmente,
    # delegamos todo a la fachada.
    facade = GameStartFacade.new()
    facade.name = "GameStartFacade"
    add_child(facade)
    facade.initialize(self, hud, player)

    # Main solo escucha las señales que le interesan a través de la fachada
    facade.restart_pressed.connect(facade.start_game)
    facade.enemy_died.connect(_on_enemy_died)
    facade.boss_died.connect(_on_boss_died)
    facade.wave_cleared.connect(_on_wave_cleared)

    if player.has_signal("player_died"):
        player.connect("player_died", _on_player_died)
```

Pruebas del software





Link del codigo: https://github.com/Hekuta-S/rumbang-0.1

UNIVERSIDAD
Popular del cesar